EE465 Quiz Cheatsheet

Sources: Consensus, EthereumBlocks, EVM, SolidityTutorial, TornadoCash

Consensus

- **Consensus**: make multiple nodes stay in sync despite unreliable network and malicious behavior.
- **SMR**: clients send txs; each node keeps an append-only ordered history.
- **SMR goals**: **consistency** = same history at all honest nodes, **liveness** = every valid client tx eventually enters history.
- **Synchronous permissioned model**: fixed known nodes, PKI, shared global clock, message sent at step t arrives before $t+1$, all honest in simplest setting.

Faults and Basic Protocols

- **Fault types**: crash, omission, Byzantine. Byzantine nodes may deviate arbitrarily but cannot forge signatures.
- **Bad SMR #1**: nodes do not communicate; consistency fails if clients contact only some nodes or orders differ.
- **Bad SMR #2**: fixed leader broadcasts ordered list each round; consistency holds, liveness fails if client sends to non-leader.
- **Rotating leaders**: leader at time t is $1 + (t \bmod n)$; leader broadcasts new tx list; every node appends it. Works with honest nodes.

Byzantine Broadcast (BB)

- **Setting**: one known sender with private input $v^* \in V$.
- **Termination**: every honest node halts with some output.
- **Agreement**: all honest nodes output same value.
- **Validity**: if sender is honest, honest nodes output sender's input.
- **SMR from BB**: in each epoch, current leader proposes ordered list L^* of not-yet-included txs; run BB on L^*; append BB output.
- Liveness in this reduction is usually stated only for requests submitted to honest nodes.

BB for $f = 1$

- **Step 1**: sender signs and sends v^* to every non-sender.
- **Step 2**: each non-sender signs what it received and forwards it to all other non-senders. Missing value is treated as $\perp$.
- **Step 3**: each non-sender majority-votes over received signed messages; ties broken consistently.
- Works for $f = 1$ because honest nodes can cross-check conflicting signed messages.
- **Fails for** $f = 2$: sender + one Byzantine non-sender can split honest nodes into two groups that see different majorities.

Dolev-Strong

- Works for any f in synchronous permissioned PKI setting.
- **Convincing message at time** t: message for value v, signed first by sender, then by at least $t-1$ distinct other nodes.
- **Protocol**: time 0 sender sends signed v^*; during times $1, \ldots, f+1$, any newly convinced non-sender signs and forwards; at end output unique convinced value or $\perp$.
- Theorem: satisfies termination, agreement, validity.

Impossibility and Thresholds

- **FLM result**: in synchronous permissioned *non-PKI* setting, if $f \geq n/3$, BB with termination + agreement + validity is impossible.
- **Asynchronous network**: no shared clock, no bound on delay, every sent message eventually arrives.
- **FLP theorem**: in asynchronous permissioned PKI setting, no deterministic BA protocol can guarantee termination + agreement + validity even for $f = 1$.
- **Partial synchrony**: global stabilization time GST is unknown; after GST delays are bounded by known Δ.
- In partial synchrony, safety must always hold; liveness may only hold after GST.
- Deterministic BA exists **iff** $f < n/3$.
- **33% threshold**: honest nodes can wait only for $n - f$ messages; those may include Byzantine nodes, so honest majority requires

$$f < \frac{n-f}{2} \iff f < \frac{n}{3} \iff n \geq 3f + 1.$$

- **CAP principle**: cannot have consistency, availability, and partition tolerance simultaneously.
- Tendermint/PBFT sacrifice liveness during partition; Bitcoin sacrifices consistency because long reorgs are possible.

Ethereum Blocks

- Ethereum launched as PoW in July 2015; switched to PoS in Sept. 2022 (**the Merge**).
- **Execution client**: executes txs and updates world state.
- **Beacon chain client**: runs PoS consensus on execution blocks.

Eth1 Header

- Main fields:
  - hashes: `parentHash`, `ommersHash`, `stateRoot`, `transactionsRoot`, `receiptsRoot`
  - others: `beneficiary`, `logsBloom`, `difficulty`, `number`, `gasLimit`, `gasUsed`, `timestamp`, `extraData`, `mixHash`, `nonce`
- Block can be written as (Header, Transactions, Uncle Headers).

Uncles / Ommers and GHOST

- **Ommer**: sibling of parent; Ethereum rewarded some stale blocks because short block times increase stale rate.
- Uncle txs are invalid; creator gets only fraction of reward and no tx fees.
- **GHOST**: choose the child whose subtree is heaviest at every fork, not merely longest chain.
- Ethereum 2.0 uses an LMD-GHOST variant.

Eth2 Execution Header

- Same major fields as Eth1, but key changes:
- `ommersHash` = hash of empty list.
- `difficulty` = 0.
- `nonce` = 8 zero bytes.
- `mixHash` replaced by `prevRandao`.
- New fee field: `baseFeePerGas`.

High-yield Header Meanings

- `parentHash`: Keccak-256 of parent header.
- `beneficiary`: destination address of block reward and tx fees.
- `stateRoot`: root of world-state trie after executing all block txs.
- `transactionsRoot`: root of trie holding all block txs.
- `receiptsRoot`: root of tx receipts trie.
- `number`: number of ancestor blocks.
- `timestamp`: Unix time of block creation.
- `extraData`: arbitrary data; validators identify themselves here.

Receipts, Logs, and Bloom

- Receipts trie is a **Merkle Patricia Trie**.
- Key = RLP encoding of tx index in block.
- Value = RLP-encoded receipt

```
[status, cumulativeGasUsed, logsBloom, logs].
```

- Log structure:

```
[address, [topic0, topic1, topic2, topic3], data].
```

- `topic0` = Keccak hash of event signature; `topic1-3` = indexed params.
- Example for `Transfer(address,address,uint256)`:
- `topic0 = keccak256("Transfer(address,address,uint256)")`.
- `topic1 = sender`, `topic2 = receiver`, `data = value`.
- **logsBloom**: OR of receipt bloom filters. Logger address and indexed topics set 3 of 2048 bits.
- Light clients use bloom filters to search relevant logs.

Gas Fields and EIP-1559

- `gasUsed`: total gas used by all block txs.
- `gasLimit`: max gas in block; slides state current value 60 million.
- `baseFeePerGas`: minimum required gas price, burned by protocol.
- Gas target is half the gas limit: 30 million.
- EIP-1559 update:

$$\text{gasTarget} = \frac{\text{gasLimit}}{2}$$

$$\delta = \frac{\text{gasUsed} - \text{gasTarget}}{4\,\text{gasLimit}} \times \text{baseFeePerGas}$$

$$\text{baseFeePerGas}_{new} = \text{baseFeePerGas} + \delta$$

- Users may add a **priority fee** (tip).

EVM

- Stack-based machine with 32-byte words; contract code stored in virtual ROM.

- Stack and memory are **volatile**; storage persists across transactions.
- Stack size limit: **1024 words**.
- Memory is a word-addressed byte array.
- Storage is a word-to-word key-value map for the current contract.

Access and Resources

- Current contract storage via `SLOAD`, `SSTORE`.
- Other contracts' storage cannot be read directly.
- Other contracts' code can be read with `EXTCODECOPY`.
- Transaction calldata via `CALLDATALOAD`, `CALLDATACOPY`.

Gas Numbers

- Base tx cost: **21000**.
- `ADD`, `SUB`: 3 gas.
- `MUL`, `DIV`: 5 gas.
- `MLOAD`, `MSTORE`: 3+ memory expansion cost.
- `SLOAD`: 2100 gas for cold key, 100 for warm key.
- `CALL`: 2600 gas for cold address, 100 for warm address.
- `BALANCE`: 2600 cold, 100 warm.
- `CREATE`: minimum 32000.
- Calldata: 4 gas per zero byte, 16 gas per non-zero byte.
- `KECCAK256`: $30 + 6$ per hashed word.
- Precompile `ecPairing` at address `0x08`: 45000 gas.

Memory Expansion

- Memory grows in 32-byte words.
- For `MSTORE`, top stack word is offset, next word is data.
- If new offset exceeds previously touched memory, expansion cost applies.

$$\text{memory_size_word} = \frac{\text{memory_byte_size} + 31}{32}$$

$$\text{memory_cost} = \frac{\text{memory_size_word}^2}{512} + 3 \cdot \text{memory_size_word}$$

Solidity

- Statically typed language for Ethereum smart contracts; JavaScript-like syntax.
- **Local vars**: inside functions, not stored in world state.
- **State vars**: outside functions, stored in world state.
- **Special vars**: e.g. `block.number`, `msg.sender`.

Core Types and Storage Forms

- Primitive types: `bool`, `uint`, `int`, `address`, `bytes1` to `bytes32`, `string`.
- **constant**: fixed at compile time; cheaper because hardcoded.
- **immutable**: set in constructor, unchangeable after deployment.
- **Mappings**: `mapping(keyType => valueType)`.
- Mapping lookup always returns a value; unset key gives default value.
- Key type cannot be reference type; value type can be any type, including mapping.
- Nested mapping example: `mapping(address => mapping(uint => bool))`.
- Arrays can be dynamic or fixed-size; use `push()` and `pop()`.
- Structs group fields; use `storage` reference for in-place updates.

Function Facts

- **view**: reads state but does not modify it.
- **pure**: neither reads nor modifies state.
- Visibility:
  - `public`: callable by contracts and EOAs.
  - `private`: only inside defining contract.
  - `internal`: defining contract + children.
  - `external`: callable from other contracts / EOAs.
- Reference-type data locations in function signatures are mandatory: `memory`, `storage`, `calldata`.
- `calldata` is read-only and cheaper when applicable.
- Public functions cannot take or return mappings.
- Functions can return multiple values.

Events, Checks, and Modifiers

- Example event: `Log(sender, message)` with sender indexed.
- Up to **3 indexed parameters**.
- Contract address, event signature, and indexed params contribute to bloom filters.
- `require(cond, "msg")` checks user input / preconditions.
- `assert(cond)` checks conditions that should never fail.
- Both revert state changes on failure.
- Modifiers run code before/after a function; `_` marks where function body runs.
- Typical modifier: `onlyOwner`.

Inheritance, Payable, and Calls

- Base function must be `virtual`; child implementation must be `override`.
- Multiple inheritance is allowed; parent search is **right to left**; `super` calls immediate parent implementation.
- State variables are not overridden directly; child re-initializes them, usually in constructor.
- `payable` functions and addresses can receive Ether.
- `address(this).balance` = Ether balance of current contract.
- Calling typed contract: `Callee(_addr).setX(x)`.
- Low-level call uses **addr.call(...)** together with **abi.encodeWithSignature(...)**.
- Interfaces: declarations only, no state vars, all functions `external`, may declare events and errors.

receive / fallback

- `receive()` handles plain Ether transfer.
- `fallback()` runs when:
  - called function does not exist, or
  - Ether sent and `receive` does not exist, or
  - Ether sent with non-empty `msg.data`.
- Fallback is often used for forwarding / proxy behavior.

Re-Entrancy and Tokens

DAO Attack

- Vulnerable pattern: external Ether transfer happens *before* balance reset.
- Attacker re-enters from fallback and calls `withdraw()` again before state updates.
- Fix 1: **checks-effects-interactions**: set balance to zero before transfer.
- Fix 2: re-entrancy lock: `require(!locked); locked=true; ...; locked=false;`

ERC-20

- Standard interface for fungible tokens.
- Events: `Transfer`, `Approval`.
- Main functions:
  - `totalSupply`, `balanceOf`, `transfer`
  - `allowance`, `approve`, `transferFrom`
- Common storage:
  - `totalSupply`, `name`, `symbol`
  - `balanceOf[address]`
  - `allowance[owner][spender]`
- `_mint(to, amount)` increases recipient balance + total supply, emits `Transfer(address(0), to, amount)`.

ERC-721 / NFTs

- NFT = non-fungible token; ownership commonly stored as `tokenId -> owner`.
- Mint checks: recipient not zero address, token id not already minted.
- NFT image is usually *not* stored on-chain; contract stores metadata link via `tokenURI()`.

Tornado Cash

- Purpose: break the visible link between deposit address and withdrawal address.
- Desired properties:
  - **soundness**: only past depositors can withdraw, and only once.
  - **privacy**: withdrawal should not be linkable to a specific deposit.

Deposit

- Private note = header + **31-byte nullifier** + **31-byte secret**.
- These 62 bytes are randomly generated on the user's computer.
- Commitment = Pedersen hash of the 62-byte secret material.
- Commitment must be fresh: `require(!commitments[c], "...")`.
- Commitment inserted into Merkle tree of depth **20**.
- No deletions $\Rightarrow$ max deposits $= 2^{20}$.
- Contract records commitment, checks `msg.value == denomination`, emits `Deposit(commitment, insertedIndex, timestamp)`.

Withdraw

- Function takes `_proof`, `_root`, `_nullifierHash`, `_recipient`, `_relayer`, `_fee`.
- SNARK statement: user knows secret + nullifier for a commitment in tree root `_root`, and `_nullifierHash` is Pedersen hash of the nullifier.
- Prevent double spend by requiring an unseen `nullifierHash`.
- Root must be one of the **last 100 Merkle roots**.
- Contract verifies proof on-chain, stores nullifier hash as spent, sends
  - `denomination - fee` to recipient
  - `fee` to relayer
- Proof also binds recipient, relayer, and fee to prevent tampering.
- Verifier contract is generated using `circom`.

Historical Note

- OFAC sanctioned Tornado Cash addresses on Aug. 8, 2022.
- Repositories were later restored for educational use; major legal cases against developers continued.

Ultra-Short Recall

- **Consensus**: consistency vs liveness.
- **BB**: termination, agreement, validity.
- **Threshold**: $f < n/3$, equivalently $n \geq 3f + 1$.
- **Async deterministic BA**: impossible (FLP).
- **Eth2 changes**: empty `ommersHash`, `difficulty=0`, `nonce=0`, `prevRandao`, `baseFeePerGas`.
- **Receipt**: `[status, cumulativeGasUsed, logsBloom, logs]`.
- **Transfer topic0**: Keccak hash of the ERC-20 Transfer event signature.
- **EVM stack**: 1024 words.
- **Base tx gas**: 21000.
- **SLOAD**: 2100 cold, 100 warm.
- **Calldata**: 4 zero byte, 16 non-zero byte.
- **Data locations**: `memory`, `storage`, `calldata`.
- **Visibility**: `public`, `private`, `internal`, `external`.
- **Re-entrancy fix**: update state before external call.
- **Tornado note**: 31-byte nullifier + 31-byte secret.
- **Tornado tree**: depth 20, max 2^{20} deposits, roots remembered: 100.